

Must-Answer Interview Questions: OOP and LLD

Use this as a placement preparation checklist. Try answering each question out loud, then practice explaining it with examples, trade-offs, and class diagrams where useful.

OOP Fundamentals

1. What is Object-Oriented Programming?
2. Why do we use OOP instead of only procedural programming?
3. What is a class?
4. What is an object?
5. What is the difference between a class and an object?
6. What are identity, state, and behavior of an object?
7. What is a reference variable?
8. What is the difference between an object and a reference?
9. What happens in memory when an object is created?
10. What is the difference between stack memory and heap memory?
11. What is a constructor?
12. Why do we need constructors?
13. What is a default constructor?
14. What is a parameterized constructor?
15. What is a copy constructor?
16. Can a constructor be private?
17. When would you use a private constructor?
18. Can constructors be overloaded?
19. Can constructors be inherited?
20. Can constructors be overridden?
21. What is a destructor or finalizer?
22. What is garbage collection?
23. How does object lifetime work in Java?
24. What is the difference between class variables and instance variables?
25. What is the difference between class methods and instance methods?
26. What is the role of the `this` keyword?
27. What is the role of the `super` keyword?
28. What are access modifiers?
29. What is the difference between public, private, protected, and default access?
30. Why should data members usually be private?

Four Pillars of OOP

1. What are the four pillars of OOP?
2. What is encapsulation?
3. What is abstraction?
4. What is inheritance?
5. What is polymorphism?

6. How is encapsulation different from abstraction?
7. How is abstraction different from inheritance?
8. How is polymorphism related to inheritance?
9. Can you explain all four pillars using one real-world example?
10. Which OOP pillar improves security?
11. Which OOP pillar improves code reuse?
12. Which OOP pillar helps hide implementation details?
13. Which OOP pillar allows one interface to support multiple behaviors?
14. Can OOP exist without inheritance?
15. Can OOP exist without polymorphism?

Encapsulation

1. What is encapsulation?
2. Why is encapsulation important?
3. How do getters and setters support encapsulation?
4. Are getters and setters always good design?
5. What is data hiding?
6. What is the difference between encapsulation and data hiding?
7. How does encapsulation reduce coupling?
8. How does encapsulation improve maintainability?
9. Give an example where direct field access is harmful.
10. How would you design a `BankAccount` class using encapsulation?
11. How do validation rules fit into encapsulation?
12. Can a class be immutable and encapsulated?
13. What is a leaky abstraction in the context of encapsulation?
14. What is the problem with exposing internal collections directly?
15. How can you safely expose a collection from a class?

Abstraction

1. What is abstraction?
2. Why is abstraction important in software design?
3. What is the difference between abstract class and interface?
4. When should you use an abstract class?
5. When should you use an interface?
6. Can an abstract class have constructors?
7. Can an abstract class have concrete methods?
8. Can an interface have default methods?
9. Can an interface contain state?
10. Can a class implement multiple interfaces?
11. Can a class extend multiple classes?
12. How does abstraction help in LLD interviews?
13. How do abstract methods work?
14. What is implementation hiding?
15. How is abstraction used in payment systems?
16. How is abstraction used in notification systems?

17. How is abstraction used in logging frameworks?
18. What is the difference between "what an object does" and "how it does it"?
19. How would you abstract different vehicle types in a parking lot system?
20. What makes an abstraction too broad or too weak?

Inheritance

1. What is inheritance?
2. Why do we use inheritance?
3. What is single inheritance?
4. What is multilevel inheritance?
5. What is hierarchical inheritance?
6. What is multiple inheritance?
7. Why does Java not support multiple inheritance using classes?
8. How do interfaces solve some multiple inheritance problems?
9. What is the diamond problem?
10. What is method overriding?
11. What is constructor chaining?
12. What is the order of constructor execution in inheritance?
13. What is the difference between **is-a** and **has-a** relationships?
14. When should inheritance be avoided?
15. Why is composition often preferred over inheritance?
16. What is fragile base class problem?
17. What is the difference between subclass and superclass?
18. Can private members be inherited?
19. Can static methods be inherited?
20. Can static methods be overridden?
21. What is method hiding?
22. What is final class?
23. What is final method?
24. Can an abstract class be final?
25. What are the risks of deep inheritance hierarchies?
26. How would you model different employee types using inheritance?
27. How would you decide whether **Car** should inherit from **Vehicle**?
28. How would you avoid inheritance misuse in a shape hierarchy?
29. What is behavioral subtyping?
30. How does inheritance affect testing?

Polymorphism

1. What is polymorphism?
2. What are the types of polymorphism?
3. What is compile-time polymorphism?
4. What is runtime polymorphism?
5. What is method overloading?
6. What is method overriding?
7. What is the difference between overloading and overriding?

8. Can method overloading depend only on return type?
9. Can overloaded methods have different access modifiers?
10. Can static methods be overloaded?
11. Can static methods be overridden?
12. Can private methods be overridden?
13. Can final methods be overridden?
14. Can constructors be overloaded?
15. Can constructors be overridden?
16. What is dynamic method dispatch?
17. What is early binding?
18. What is late binding?
19. How does runtime polymorphism work internally?
20. What is upcasting?
21. What is downcasting?
22. What is the risk of downcasting?
23. What is the difference between reference type and object type?
24. How does polymorphism help in writing extensible code?
25. Give an example of polymorphism in a payment system.
26. Give an example of polymorphism in a notification system.
27. Give an example of polymorphism in a game design.
28. How does polymorphism support Open/Closed Principle?
29. What is operator overloading?
30. Why does Java not support custom operator overloading?

Association, Aggregation, and Composition

1. What is association?
2. What is aggregation?
3. What is composition?
4. What is the difference between association and aggregation?
5. What is the difference between aggregation and composition?
6. What is the difference between inheritance and composition?
7. What is a **has-a** relationship?
8. What is a **uses-a** relationship?
9. What is a dependency relationship?
10. What is object ownership?
11. How does object lifetime differ in aggregation and composition?
12. Is **Department** and **Professor** aggregation or composition?
13. Is **House** and **Room** aggregation or composition?
14. Is **Car** and **Engine** aggregation or composition?
15. Is **Library** and **Book** aggregation or composition?
16. Is **Order** and **OrderItem** aggregation or composition?
17. Is **Team** and **Player** aggregation or composition?
18. How do you show association in a UML class diagram?
19. How do you show aggregation in a UML class diagram?
20. How do you show composition in a UML class diagram?

Interfaces and Abstract Classes

1. What is an interface?
2. What is an abstract class?
3. What are the differences between interface and abstract class?
4. Can an interface extend another interface?
5. Can an abstract class implement an interface?
6. Can an interface have private methods?
7. Can an interface have static methods?
8. Can an abstract class have static methods?
9. Can an interface have constructors?
10. Can an abstract class have constructors?
11. When would you choose an interface over an abstract class?
12. When would you choose an abstract class over an interface?
13. What is marker interface?
14. What is functional interface?
15. What is interface segregation?
16. What is default method conflict in interfaces?
17. How can interfaces help in unit testing?
18. How can interfaces reduce coupling?
19. How do interfaces help with dependency inversion?
20. How would you design a common interface for multiple payment modes?

SOLID Principles

1. What does SOLID stand for?
2. What is Single Responsibility Principle?
3. What is Open/Closed Principle?
4. What is Liskov Substitution Principle?
5. What is Interface Segregation Principle?
6. What is Dependency Inversion Principle?
7. Why are SOLID principles important in LLD?
8. Give an example of SRP violation.
9. Give an example of OCP violation.
10. Give an example of LSP violation.
11. Give an example of ISP violation.
12. Give an example of DIP violation.
13. How does SRP improve maintainability?
14. How does OCP help add new features?
15. How does LSP protect polymorphism?
16. How does ISP prevent fat interfaces?
17. How does DIP reduce coupling?
18. What is the difference between dependency inversion and dependency injection?
19. Can following SOLID too strictly make code worse?
20. How would you apply SOLID to a notification system?
21. How would you apply SOLID to a payment system?
22. How would you apply SOLID to a parking lot system?

23. How would you refactor a class that violates SRP?
24. How would you refactor a switch-case that violates OCP?
25. How would you detect an LSP violation in subclasses?
26. How would you split a large interface using ISP?
27. How would you invert dependencies in a service class?
28. Which SOLID principle is most related to abstraction?
29. Which SOLID principle is most related to polymorphism?
30. Which SOLID principle is easiest to overuse?

Design Patterns

1. What is a design pattern?
2. Why are design patterns useful?
3. What are creational design patterns?
4. What are structural design patterns?
5. What are behavioral design patterns?
6. What is Singleton pattern?
7. What are the problems with Singleton?
8. How do you make Singleton thread-safe?
9. What is Factory Method pattern?
10. What is Abstract Factory pattern?
11. What is Builder pattern?
12. What is Prototype pattern?
13. What is Adapter pattern?
14. What is Decorator pattern?
15. What is Strategy pattern?
16. What is Observer pattern?
17. What is Command pattern?
18. What is State pattern?
19. What is Template Method pattern?
20. What is Chain of Responsibility pattern?
21. What is Facade pattern?
22. What is Proxy pattern?
23. What is Composite pattern?
24. What is Iterator pattern?
25. What is MVC pattern?
26. What is Repository pattern?
27. What is Dependency Injection pattern?
28. What is the difference between Factory and Abstract Factory?
29. What is the difference between Strategy and State?
30. What is the difference between Decorator and Inheritance?
31. What is the difference between Adapter and Facade?
32. What is the difference between Observer and Publisher-Subscriber?
33. When would you use Builder instead of constructor overloading?
34. When would you use Strategy instead of if-else?
35. When would you use Factory instead of direct object creation?
36. What design pattern would you use for a logging framework?

37. What design pattern would you use for a notification system?
38. What design pattern would you use for a vending machine?
39. What design pattern would you use for an elevator system?
40. What design pattern would you use for a chess game?

UML and Class Diagrams

1. What is UML?
2. Why are UML diagrams used in LLD?
3. What is a class diagram?
4. What are the components of a class diagram?
5. How do you represent a class in UML?
6. How do you represent attributes in UML?
7. How do you represent methods in UML?
8. How do you represent public, private, and protected visibility?
9. How do you represent inheritance?
10. How do you represent interface implementation?
11. How do you represent association?
12. How do you represent aggregation?
13. How do you represent composition?
14. How do you represent dependency?
15. How do you represent multiplicity?
16. What does one-to-one multiplicity mean?
17. What does one-to-many multiplicity mean?
18. What does many-to-many multiplicity mean?
19. How would you draw a UML diagram for a parking lot?
20. How would you draw a UML diagram for an elevator system?
21. How would you draw a UML diagram for a library system?
22. How would you draw a UML diagram for a food delivery system?
23. How would you draw a UML diagram for an ATM?
24. How would you decide which classes to include in a UML diagram?
25. How much detail should be shown in an interview UML diagram?

LLD Fundamentals

1. What is Low Level Design?
2. How is LLD different from High Level Design?
3. What is the goal of an LLD interview?
4. What steps do you follow in an LLD interview?
5. How do you gather requirements before designing?
6. What are functional requirements?
7. What are non-functional requirements?
8. Why should you clarify assumptions before designing?
9. How do you identify main entities in a system?
10. How do you identify relationships between entities?
11. How do you identify responsibilities of classes?
12. How do you decide which methods belong in which class?

13. How do you avoid creating a god class?
14. How do you decide between inheritance and composition?
15. How do you decide between interface and abstract class?
16. How do you design for extensibility?
17. How do you design for testability?
18. How do you design for maintainability?
19. How do you handle changing requirements in LLD?
20. How do you explain trade-offs in an LLD interview?
21. What is domain modeling?
22. What is responsibility-driven design?
23. What is separation of concerns?
24. What is cohesion?
25. What is coupling?
26. What is high cohesion and low coupling?
27. What is a service class?
28. What is a manager class?
29. What is a controller class?
30. What is a repository class?
31. What is a model or entity class?
32. What is a value object?
33. What is an enum and when should you use it?
34. What is a DTO?
35. What is the difference between entity and DTO?
36. What is the difference between service and repository?
37. What is the difference between controller and service?
38. What is the difference between domain logic and application logic?
39. How do you keep business rules out of controllers?
40. How do you decide package structure for an LLD solution?

Common LLD Interview Problems

1. Design a parking lot system.
2. Design an elevator system.
3. Design a library management system.
4. Design an ATM.
5. Design a vending machine.
6. Design a chess game.
7. Design a snake and ladder game.
8. Design a tic-tac-toe game.
9. Design a splitwise-like expense sharing system.
10. Design a bookmyshow-like movie ticket booking system.
11. Design a food delivery system.
12. Design a ride sharing system.
13. Design a hotel management system.
14. Design a railway reservation system.
15. Design an airline reservation system.
16. Design a car rental system.

17. Design a meeting scheduler.
18. Design a calendar application.
19. Design a notification system.
20. Design a logging framework.
21. Design a rate limiter.
22. Design a cache.
23. Design an LRU cache.
24. Design a file system.
25. Design a text editor.
26. Design a traffic signal system.
27. Design a coffee machine.
28. Design a restaurant management system.
29. Design an online shopping cart.
30. Design an inventory management system.
31. Design a wallet system.
32. Design a payment gateway.
33. Design an order management system.
34. Design a classroom management system.
35. Design a quiz platform.
36. Design a social media feed.
37. Design a messaging app.
38. Design a job portal.
39. Design a hospital management system.
40. Design a cricket scorecard system.

Parking Lot LLD Questions

1. What are the main requirements of a parking lot system?
2. What types of vehicles should the parking lot support?
3. What types of parking spots should exist?
4. How do you assign a parking spot to a vehicle?
5. How do you release a parking spot?
6. How do you calculate parking fees?
7. How do you support multiple entry and exit gates?
8. How do you track available spots?
9. How do you represent parking tickets?
10. How do you handle lost tickets?
11. How do you support different pricing strategies?
12. How do you design for multiple floors?
13. How do you handle electric vehicle charging spots?
14. Which design patterns can be used in parking lot design?
15. Which SOLID principles are important in parking lot design?

Elevator System LLD Questions

1. What are the main requirements of an elevator system?
2. What are the main classes in an elevator system?

3. How do you represent elevator states?
4. How do you represent direction?
5. How do you handle internal and external requests?
6. How do you assign elevators to requests?
7. How do you minimize waiting time?
8. How do you handle multiple elevators?
9. How do you handle emergency stop?
10. How do you handle overload?
11. How do you handle maintenance mode?
12. What scheduling algorithms can be used?
13. Which design patterns can be used in elevator design?
14. How would you make the design extensible for new scheduling strategies?
15. How would you test an elevator system design?

Vending Machine LLD Questions

1. What are the main requirements of a vending machine?
2. What states can a vending machine have?
3. How do you represent inventory?
4. How do you accept coins or notes?
5. How do you return change?
6. How do you dispense an item?
7. How do you handle insufficient balance?
8. How do you handle out-of-stock items?
9. How do you cancel a transaction?
10. How do you refund money?
11. How would you use the State pattern?
12. How would you add digital payments?
13. How would you support multiple item categories?
14. How would you handle machine maintenance?
15. How would you test the vending machine design?

ATM LLD Questions

1. What are the main requirements of an ATM?
2. What are the main classes in an ATM system?
3. How do you authenticate a user?
4. How do you validate a card?
5. How do you validate a PIN?
6. How do you handle cash withdrawal?
7. How do you handle balance enquiry?
8. How do you handle deposit?
9. How do you handle fund transfer?
10. How do you represent ATM states?
11. How do you handle insufficient ATM cash?
12. How do you handle insufficient account balance?
13. How do you dispense notes of different denominations?

14. How do you handle transaction history?
15. How do you secure sensitive operations?

Library Management LLD Questions

1. What are the main requirements of a library management system?
2. What are the main entities in a library system?
3. How do you model books and book copies?
4. How do you model members?
5. How do you issue a book?
6. How do you return a book?
7. How do you reserve a book?
8. How do you calculate fines?
9. How do you search books?
10. How do you handle multiple copies of the same book?
11. How do you handle unavailable books?
12. How do you handle membership limits?
13. How do you notify users about due dates?
14. How do you design extensible search filters?
15. Which design patterns can be used in this system?

BookMyShow LLD Questions

1. What are the main requirements of a movie ticket booking system?
2. What are the main entities in BookMyShow?
3. How do you model cities, theatres, screens, shows, and seats?
4. How do you show available seats?
5. How do you reserve seats temporarily?
6. How do you prevent double booking?
7. How do you handle payment failure?
8. How do you release locked seats?
9. How do you generate a ticket?
10. How do you cancel a booking?
11. How do you support different seat types?
12. How do you support dynamic pricing?
13. How do you handle concurrency in booking?
14. Which design patterns can be used?
15. How would you test this design?

Splitwise LLD Questions

1. What are the main requirements of an expense sharing system?
2. What are the main entities in Splitwise?
3. How do you represent users and groups?
4. How do you represent expenses?
5. How do you split expenses equally?
6. How do you split expenses by percentage?
7. How do you split expenses by exact amount?

8. How do you validate split amounts?
9. How do you simplify balances?
10. How do you show who owes whom?
11. How do you settle expenses?
12. How do you support multiple currencies?
13. How do you store transaction history?
14. Which design patterns can be used?
15. How would you extend the system for recurring expenses?

Chess LLD Questions

1. What are the main requirements of a chess game?
2. What are the main classes in chess design?
3. How do you represent a board?
4. How do you represent cells or squares?
5. How do you represent pieces?
6. How do you validate moves for each piece?
7. How do you handle turn management?
8. How do you detect check?
9. How do you detect checkmate?
10. How do you detect stalemate?
11. How do you handle castling?
12. How do you handle pawn promotion?
13. How do you handle en passant?
14. How do you track captured pieces?
15. Which design patterns can be used?

Snake and Ladder LLD Questions

1. What are the main requirements of a snake and ladder game?
2. What are the main entities in the game?
3. How do you represent the board?
4. How do you represent snakes and ladders?
5. How do you represent players?
6. How do you represent dice?
7. How do you handle turns?
8. How do you handle multiple dice?
9. How do you determine the winner?
10. How do you handle exact winning position rules?
11. How do you generate random dice values?
12. How do you make the design testable despite randomness?
13. How would you support different board sizes?
14. How would you support multiple players?
15. Which design patterns can be used?

Tic-Tac-Toe LLD Questions

1. What are the main requirements of tic-tac-toe?

2. What are the main classes in tic-tac-toe?
3. How do you represent the board?
4. How do you validate a move?
5. How do you switch turns?
6. How do you detect a winner?
7. How do you detect a draw?
8. How do you support different board sizes?
9. How do you support different win conditions?
10. How do you design a bot player?
11. How do you separate game logic from UI?
12. How do you test winner detection?
13. How do you model player symbols?
14. How would you design undo functionality?
15. Which design patterns can be used?

LLD Design Quality Questions

1. How do you identify a god class?
2. How do you break a god class into smaller classes?
3. How do you identify high coupling?
4. How do you reduce coupling?
5. How do you identify low cohesion?
6. How do you improve cohesion?
7. How do you decide class responsibilities?
8. How do you name classes in LLD?
9. How do you name methods in LLD?
10. How do you keep a design simple?
11. How do you avoid over-engineering?
12. How do you handle future requirements without guessing too much?
13. How do you design extensible enums?
14. When should an enum be replaced by polymorphism?
15. When should a switch-case be replaced by Strategy pattern?
16. When is switch-case acceptable in LLD?
17. How do you design validation logic?
18. Where should business rules live?
19. Where should object creation logic live?
20. How do you design error handling?
21. How do you design logging?
22. How do you design configuration?
23. How do you design retry behavior?
24. How do you design id generation?
25. How do you design audit history?
26. How do you make code easy to test?
27. How do you mock external dependencies?
28. How do you avoid circular dependencies?
29. How do you review your own LLD solution?
30. How do you explain your design clearly to an interviewer?

Java OOP Interview Questions

1. Is Java a pure object-oriented language?
2. Why is Java not considered fully pure OOP?
3. What is the difference between primitive types and objects?
4. What is autoboxing?
5. What is unboxing?
6. What is the difference between `==` and `equals()`?
7. What is the contract between `equals()` and `hashCode()`?
8. Why should `hashCode()` be overridden when `equals()` is overridden?
9. What is the difference between `String`, `StringBuilder`, and `StringBuffer`?
10. Why is `String` immutable?
11. What is the difference between final variable, final method, and final class?
12. What is static keyword?
13. What is static block?
14. What is static nested class?
15. What is inner class?
16. What is anonymous class?
17. What is enum in Java?
18. Can enum have methods and constructors?
19. What is exception handling?
20. What is the difference between checked and unchecked exceptions?
21. What is the difference between `throw` and `throws`?
22. What is the difference between `final`, `finally`, and `finalize`?
23. What is method signature?
24. What is covariant return type?
25. What is generics?
26. What is type erasure?
27. What is a wildcard in generics?
28. What is the difference between `List<? extends T>` and `List<? super T>`?
29. What is collection framework?
30. How do OOP principles appear in Java collections?

Scenario-Based OOP Questions

1. How would you design a class for a bank account?
2. How would you design a class for a student?
3. How would you design a class for an employee payroll system?
4. How would you design a class hierarchy for vehicles?
5. How would you design a class hierarchy for shapes?
6. How would you design a payment system using polymorphism?
7. How would you design a notification system using abstraction?
8. How would you design a logger using Singleton?
9. How would you design a report generator using Strategy?
10. How would you design a document editor using Command pattern?
11. How would you design a pizza ordering system using Decorator?
12. How would you design a computer object using Builder?

13. How would you design an adapter for a third-party payment gateway?
14. How would you design undo and redo functionality?
15. How would you design a system where different users have different permissions?
16. How would you design a shopping cart?
17. How would you design coupon and discount logic?
18. How would you design order status transitions?
19. How would you design a state machine using OOP?
20. How would you refactor duplicate code across sibling classes?

Tricky OOP Questions

1. Can an object exist without a class?
2. Can a class exist without objects?
3. Can an abstract class have no abstract methods?
4. Can an interface be empty?
5. Can we instantiate an abstract class?
6. Can we instantiate an interface?
7. Can a constructor call another constructor?
8. Can a constructor be recursive?
9. Can a constructor be synchronized?
10. Can a constructor return a value?
11. Can an overridden method throw broader checked exceptions?
12. Can an overloaded method throw different exceptions?
13. Can private methods be overloaded?
14. Can private methods be overridden?
15. Can a subclass reduce visibility of an overridden method?
16. Can a subclass increase visibility of an overridden method?
17. Can static variables be polymorphic?
18. Can instance variables be overridden?
19. Can interfaces extend multiple interfaces?
20. Can abstract classes implement interfaces without implementing all methods?
21. What happens if a parent and child class have a field with the same name?
22. What happens if a parent and child class have a static method with the same signature?
23. What happens if a parent reference points to a child object?
24. Which method is called when overloaded methods are selected?
25. Which method is called when overridden methods are selected?

Interview Practice Prompts

1. Explain OOP to a beginner in two minutes.
2. Explain encapsulation with a real-world example.
3. Explain abstraction with a real-world example.
4. Explain inheritance with a real-world example.
5. Explain polymorphism with a real-world example.
6. Explain SOLID principles in five minutes.
7. Explain why composition is preferred over inheritance.
8. Explain the difference between interface and abstract class with examples.

9. Explain the difference between association, aggregation, and composition.
10. Explain the difference between overloading and overriding.
11. Explain the Liskov Substitution Principle using a bad design example.
12. Explain how you would approach any LLD problem in an interview.
13. Explain your parking lot design in five minutes.
14. Explain your elevator design in five minutes.
15. Explain your vending machine design in five minutes.
16. Explain your Splitwise design in five minutes.
17. Explain your BookMyShow design in five minutes.
18. Explain your chess design in five minutes.
19. Explain how your design handles future requirements.
20. Explain the trade-offs in one of your LLD solutions.